LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

Modul 4 Enkapsulasi



RAYMUNDUS ARIEL ABAS

103132430021

S1SD05

PROGRAM STUDI S-1 SAINS DATA

FAKULTAS DIREKTORAT KAMPUS PURWOKERTO

TELKOM UNIVERSITY

PURWOKERTO

2026

A. Dasar Teori

1. Java Modifier

Modifier dalam bahasa pemrograman Java merupakan kata kunci yang digunakan untuk menentukan hak akses dan karakteristik suatu elemen program, seperti kelas, atribut, metode, maupun konstruktor. Pada praktikum ini, terdapat tiga jenis access modifier yang dipelajari, yaitu private, public, dan protected. Dalam class diagram, modifier private direpresentasikan dengan simbol minus (-), public menggunakan simbol plus (+), sedangkan protected menggunakan simbol pagar (#).

Deklarasi modifier pada Java dapat diterapkan pada beberapa elemen program, antara lain kelas, atribut, dan metode. Penggunaan modifier tersebut menentukan sejauh mana suatu elemen dapat diakses oleh bagian program lainnya. Modifier public memungkinkan elemen diakses dari kelas mana pun selama ketentuan aksesnya terpenuhi. Sebaliknya, modifier private membatasi akses hanya pada kelas tempat elemen tersebut dideklarasikan, sedangkan protected memungkinkan akses dari kelas dalam package yang sama serta kelas turunan dengan ketentuan tertentu.

Untuk mengelola atribut yang memiliki akses terbatas, Java menyediakan metode *getter* dan *setter*. Getter digunakan untuk memperoleh nilai suatu atribut, sedangkan setter berfungsi untuk mengubah atau menetapkan nilainya. Penamaan kedua metode tersebut umumnya diawali dengan kata get dan set, kemudian diikuti nama atribut yang bersangkutan. Dengan demikian, penggunaan modifier dan metode getter-setter dapat membantu menerapkan prinsip enkapsulasi dalam pemrograman berorientasi objek.

2. Constructor

Constructor merupakan bagian khusus dalam sebuah kelas Java yang berfungsi untuk menginisialisasi objek ketika objek tersebut dibuat. Constructor memiliki nama yang sama dengan nama kelasnya dan tidak memiliki tipe nilai kembalian, termasuk void. Objek dibuat dengan menggunakan kata kunci new, yang diikuti pemanggilan constructor sesuai dengan parameter yang tersedia.

Apabila suatu kelas tidak memiliki constructor yang dideklarasikan secara eksplisit, Java akan menyediakan *default constructor* secara otomatis, selama kelas tersebut belum mendefinisikan constructor lainnya. Default constructor tidak memiliki parameter dan akan memberikan nilai awal sesuai dengan tipe data atribut yang bersangkutan apabila tidak ada inisialisasi lain. Sebagai contoh, atribut bertipe objek atau String secara default bernilai null, sedangkan atribut numerik primitif memiliki nilai awal sesuai ketentuan Java.

Selain default constructor, programmer dapat membuat constructor dengan parameter untuk memberikan nilai awal tertentu pada atribut objek. Dengan cara ini, objek dapat dibentuk dengan kondisi awal yang sesuai dengan kebutuhan program. Constructor

dapat menggunakan access modifier seperti public atau private, bergantung pada kebutuhan pengaturan akses dan proses pembuatan objek dalam aplikasi.

3. Package

Package dalam Java merupakan mekanisme untuk mengelompokkan kelas, interface, dan elemen program lainnya ke dalam suatu struktur berdasarkan fungsi atau kategorinya. Package membantu mengorganisasi kode agar lebih terstruktur, mempermudah pemeliharaan program, serta mengurangi kemungkinan konflik penamaan antarkelas.

Penggunaan package juga berkaitan dengan mekanisme akses antarkelas. Apabila dua kelas berada dalam package yang sama, kelas tersebut dapat menggunakan nama kelas lainnya secara langsung tanpa perlu menambahkan pernyataan import, selama hak aksesnya memungkinkan. Sementara itu, jika kelas yang akan digunakan berada dalam package berbeda, programmer umumnya perlu menuliskan pernyataan import agar kelas tersebut dapat dirujuk menggunakan nama sederhananya. Alternatif lainnya adalah menggunakan nama kelas yang sepenuhnya memenuhi kualifikasi package (*fully qualified name*).

Dengan memahami konsep package, programmer dapat menyusun struktur proyek Java secara lebih sistematis dan mengelola hubungan antarkelas dengan lebih mudah.

B. Guided

1. [Manusia dan Main]

Manusia:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package raymundusariel.guided1.manusia;

/**
 *
 * @author ACER
 */
public class Manusia {
    //definisi atribut ([Akses modifier] [Data Type] [Nama atribut])
    private String nama;
    private int umur;

    //4.2 constructor
    //default
    public Manusia(){};
    //bukan default
    public Manusia(String nama){
        this.nama = nama;
    }
    public Manusia(String nama, int umur){
        this.nama = nama;
        this.umur = umur;
    }

    //method setter
    public void setNama(String nama){
        this.nama = nama;
    }
    public void setUmur(int umur){
        this.umur = umur;
    }

    //method getter
    public String getNama(){
        return nama;
    }
    public int getUmur(){
        return umur;
```

```
        }
}
```

Output:Tidak ada output/bukan file main

Main as a DemoManusia

```
        /*
         * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
        default.txt to change this license
         * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
        edit this template
         */
        package raymundusariel.guided1.main;

        import raymundusariel.guided1.manusia.Manusia;

        /**
         *
         * @author ACER
         */
        public class DemoManusia {
           public static void main(String[] args){
              Manusia arrMns[] = new Manusia[3];

              //Cons pertama
              Manusia objMns1 = new Manusia();

              //Cons kedua
              Manusia objMns2 = new Manusia("Ignatius Loyola");

              //Cons ketiga
              Manusia objMns3 = new Manusia("Aloysius Gonzaga", 17);

              arrMns[0] = objMns1;
              arrMns[1] = objMns2;
              arrMns[2] = objMns3;

              for (int i=0; i<3; i++){
                 System.out.println("Nama:  " + arrMns[i].getNama());
                 System.out.println("Umur:  " + arrMns[i].getUmur());
                 System.out.println();
              }
           }
}
```

Output :

```
[
    --- exec:3.6.3:exec (default-cli) @ guided ---
    Nama:  null
    Umur:  0

    Nama:  Ignatius Loyola
    Umur:  0

    Nama:  Aloysius Gonzaga
    Umur:  17

    ------------------------------------------------------------------------
]
```

2. Guided 2:

HargaPulsa.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package raymundusariel.guided2;

/**
 *
 * @author ACER
 */
public class HargaPulsa {

    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

Output:Tidak ada output/bukan file main

HargaToken.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
```

```
        * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
        edit this template
        */
        package raymundusariel.guided2;

        /**
        *
        * @author ACER
        */
        public class HargaToken {

            public void info() {
                System.out.println("Harga Token");
                System.out.println("Rp. 20000");
                System.out.println("Rp. 50000");
                System.out.println("Rp. 100000");
                System.out.println("Rp. 200000");
                System.out.println("Rp. 500000");
            }
}
```

Output:Tidak ada output/bukan file main

Main.java:

```
        /*
        * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
        default.txt to change this license
        * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
        edit this template
        */
        package raymundusariel.guided2;

        /**
        *
        * @author ACER
        */
        public class Main {

            public static void main(String[] args) {
                HargaToken objectToken = new HargaToken();
                objectToken.info();
                HargaPulsa objectPulsa = new HargaPulsa();
                objectPulsa.info();
            }
}
```

Output :

```
[
    --- exec:3.6.3:exec (default-cli) @ guided ---
    Harga Token
    Rp. 20000
    Rp. 50000
    Rp. 100000
    Rp. 200000
    Rp. 500000
    Harga Pulsa
    Rp. 5000
    Rp. 10000
    Rp. 20000
    Rp. 50000
    Rp. 100000
    ------------------------------------------------------------------------
]
```

C. Unguided

1. Soal Unguided 1. Buatlah program Java untuk sistem pencatatan dataset sebelum dianalisis menggunakan tiga package, yaitu model, laporan, dan main. Pada package model, buat class Dataset dengan instance field nama, jumlahBaris, jumlahKolom, dan jumlahMissing, class field static final BATAS_MISSING = 5.0, serta variabel static untuk menghitung total objek. Class Dataset memiliki 3 constructor (tanpa parameter, 1 parameter nama, dan parameter lengkap), getter-setter dengan validasi nilai agar tidak negatif, method getPersentaseMissing() untuk menghitung persentase sel kosong, serta method perluDibersihkan(). On package laporan, buat class LaporanDataset dengan method cetak() untuk menampilkan laporan dataset beserta statusnya (Perlu dibersihkan atau Bersih). Di class Main pada package main, buat tiga objek Dataset menggunakan ketiga constructor berbeda untuk data Titanic (891 baris, 12 kolom, 866 missing), Wine Quality, dan Iris (150 baris, 5 kolom, 0 missing), simpan ketiganya dalam array, tampilkan laporannya menggunakan perulangan, serta tampilkan total dataset yang telah dibuat

   Jawaban:

   Dataset.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package raymundusariel.unguided.model;
```

```
/**
 *
 * @author ACER
 */
public class Dataset {
    // Atribut privat (Enkapsulasi)
    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    // Konstanta
    public static final double BATAS_MISSING = 5.0;

    // Variabel milik class untuk menghitung total objek Dataset yang dibuat
    private static int totalDataset = 0;

    // Constructor 1: Tanpa parameter
    public Dataset() {
        totalDataset++;
    }

    // Constructor 2: Hanya parameter nama
    public Dataset(String nama) {
        this.nama = nama;
        totalDataset++;
    }

    // Constructor 3: Parameter lengkap
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++;
    }

    // Method static untuk mengambil nilai totalDataset
    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter & Setter untuk nama
    public String getNama() {
```

```
    return nama;
}

public void setNama(String nama) {
    this.nama = nama;
}

// Getter & Setter untuk jumlahBaris (Validasi: nilai tidak boleh < 0)
public int getJumlahBaris() {
    return jumlahBaris;
}

public void setJumlahBaris(int jumlahBaris) {
    if (jumlahBaris >= 0) {
        this.jumlahBaris = jumlahBaris;
    }
}

// Getter & Setter untuk jumlahKolom (Validasi: nilai tidak boleh < 0)
public int getJumlahKolom() {
    return jumlahKolom;
}

public void setJumlahKolom(int jumlahKolom) {
    if (jumlahKolom >= 0) {
        this.jumlahKolom = jumlahKolom;
    }
}

// Getter & Setter untuk jumlahMissing (Validasi: nilai tidak boleh < 0)
public int getJumlahMissing() {
    return jumlahMissing;
}

public void setJumlahMissing(int jumlahMissing) {
    if (jumlahMissing >= 0) {
        this.jumlahMissing = jumlahMissing;
    }
}

// Hitung persentase sel kosong
public double getPersentaseMissing() {
    long totalSel = (long) jumlahBaris * jumlahKolom;
    if (totalSel == 0) {
        return 0.0;
    }
```

```
        return ((double) jumlahMissing / totalSel) * 100.0;
    }

    // Cek apakah dataset perlu dibersihkan
    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

Output:Tidak ada output/bukan file main

LaporanDataset.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
to change this license
 */

package raymundusariel.unguided.laporan;

import raymundusariel.unguided.model.Dataset;

/**
 *
 * @author ACER
 */
public class LaporanDataset {

    public void cetak(Dataset dataset) {
        String status = dataset.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + dataset.getNama());
        System.out.println("Jumlah Baris : " + dataset.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + dataset.getJumlahKolom());
        System.out.printf("Missing              : %d sel (%.2f%%)%n",
dataset.getJumlahMissing(), dataset.getPersentaseMissing());
        System.out.println("Status       : " + status);
        System.out.println();
    }
}
```

Output:Tidak ada output/bukan file main

Main.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template
 */
package raymundusariel.unguided.main;

import raymundusariel.unguided.laporan.LaporanDataset;
import raymundusariel.unguided.model.Dataset;

/**
 *
 * @author ACER
 */
public class Main {
    public static void main(String[] args) {

        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        Dataset d2 = new Dataset("Wine Quality");
        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        Dataset[] daftarDataset = {d1, d2, d3};

        LaporanDataset laporan = new LaporanDataset();

        for (Dataset d : daftarDataset) {
            laporan.cetak(d);
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ unguided ---
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8,10%)
Status       : Perlu dibersihkan

=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0,00%)
Status       : Bersih

=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0,00%)
Status       : Bersih

Total dataset dibuat : 3
------------------------------------------------------------------------
```

Penjelasan Alur Program

Program Sistem Pencatatan Dataset dirancang untuk mengelola atribut data serta mengevaluasi proporsi nilai hilang sebelum proses analisis data dilakukan. Struktur program dibagi ke dalam tiga package utama, yaitu model, laporan, dan main, guna mewujudkan arsitektur kode yang modular dan menerapkan prinsip enkapsulasi secara menyeluruh.

Mekanisme kerja program berjalan melalui beberapa tahapan berikut:

1. Penyimpanan State dan Konfigurasi: Kelas Dataset mengamankan data internal melalui atribut privat, mencakup nama dataset, jumlah baris, jumlah kolom, dan total entri kosong. Ambang batas toleransi nilai hilang didefinisikan secara konstan melalui variabel static final BATAS_MISSING sebesar 5.0%, sedangkan variabel static totalDataset dialokasikan untuk melacak akumulasi objek yang telah terbentuk di memori.
2. Instansiasi Objek: Penyusunan objek Dataset didukung oleh mekanisme constructor overloading melalui tiga pilihan konstruktor: tanpa parameter, satu parameter nama, dan parameter lengkap. Setiap pemanggilan konstruktor akan menaikkan nilai counter pada variabel static totalDataset.
3. Proteksi Mutasi Data: Akses pembacaan dan modifikasi data dikontrol melalui metode getter dan setter. Setter untuk dimensi baris, kolom, serta jumlah data hilang dilengkapi logika validasi non-negatif guna mencegah anomali data bernilai di bawah nol.

4. Komputasi Kualitas Data: Metode getPersentaseMissing() mengkalkulasi rasio sel kosong terhadap kapasitas total sel dataset. Selanjutnya, metode perluDibersihkan() membandingkan persentase tersebut terhadap batas toleransi BATAS_MISSING untuk menentukan kelayakan dataset.
5. Generasi Output Laporan: Kelas LaporanDataset memanfaatkan metode cetak() untuk mengekstrak metrik dari objek Dataset, lalu menyajikannya secara terformat ke terminal, mencakup ringkasan dimensi, persentase nilai hilang, serta rekomendasi status kebersihan data.
6. Eksekusi Utama: Kelas Main menginisialisasi tiga instansiasi data (Titanic, Wine Quality, dan Iris) menggunakan variasi konstruktor yang tersedia. Ketiga objek dihimpun ke dalam sebuah array bertipe Dataset, diproses melalui iterasi perulangan untuk mencetak laporan beruntun, lalu ditutup dengan pemanggilan metode static getTotalDataset() guna memverifikasi seluruh objek yang berhasil diinisialisasi.

D. Kesimpulan

Praktikum ini menegaskan penerapan pilar dasar Pemrograman Berorientasi Objek pada Java, terutama enkapsulasi, restriksi hak akses, serta pengorganisasian kode berbasis package. Melalui implementasi Sistem Pencatatan Dataset, pemisahan komponen ke dalam package model, laporan, dan main berhasil menjaga keteraturan struktur direktori sekaligus membatasi dependensi antarkelas. Proteksi data diterapkan secara ketat dengan menetapkan modifier private pada atribut kelas Dataset, yang mutasinya dikontrol melalui metode getter-setter berlogika validasi. Di samping itu, praktikum ini mendemonstrasikan fleksibilitas instansiasi objek melalui *constructor overloading*, penggunaan *static keyword* untuk pengelolaan data global secara independen dari instans, serta pemanfaatan array objek untuk otomasi pelaporan data yang efisien.